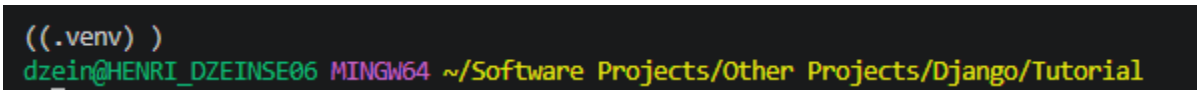
To use Django

1. Install it
2. Run Virtual Environment to be able to run Django
3. Do the Tutorial

https://www.youtube.com/watch?v=GZbeL5AcTgw

Run these commands

1. python -m venv .venv OR py -3 -m venv .venv
2. Check your folder structure, you should now see the .venv folder in your folder structure. You can now kill the terminal/bash whatever it is (by running exit command) and then open it again.
3. The indication that you are in the virtual environment (VE) should be (venv) before your file structure on the terminal
    a. Example:



    b. ((.venv)) tells us we're in the VE
4. To leave the VE run: deactivate
5. To reactivate, just exit the terminal/bash by running "exit"

Creating a project

1. Run this command in the terminal: django-admin startproject mysite djangotutorial
2. Then cd into djangotutorial or whatever name you gave it

Understanding the files and structure

```
djangotutorial/
    manage.py
    mysite/
        __init__.py
        settings.py
        urls.py
        asgi.py
        wsgi.py
```

- **`manage.py`**: A command-line utility that lets you interact with this Django project in various ways. You can read all the details about **`manage.py`** in django-admin and manage.py.
- **`mysite/`**: A directory that is the actual Python package for your project. Its name is the Python package name you'll need to use to import anything inside it (e.g. **`mysite.urls`**).
- **`mysite/__init__.py`**: An empty file that tells Python that this directory should be considered a Python package. If you're a Python beginner, read more about packages in the official Python docs.
- **`mysite/settings.py`**: Settings/configuration for this Django project. Django settings will tell you all about how settings work.
- **`mysite/urls.py`**: The URL declarations for this Django project; a "table of contents" of your Django-powered site. You can read more about URLs in URL dispatcher.
- **`mysite/asgi.py`**: An entry-point for ASGI-compatible web servers to serve your project. See How to deploy with ASGI for more details.
- **`mysite/wsgi.py`**: An entry-point for WSGI-compatible web servers to serve your project. See How to deploy with WSGI for more details.

Run the project

1. Run this command: python manage.py runserver
2. Navigate to: http://127.0.0.1:8000/ where the project is running

Apps vs Projects

- What's the difference between a project and an app? An app is a web application that does something – e.g., a blog system, a database of public records or a small poll app. A project is a collection of configuration and apps for a particular website. A project can contain multiple apps. An app can be in multiple projects.

Creating an app

1. We're creating the Poll app. We use this command: python manage.py startapp polls
2. Navigate to polls/views.py and paste this code in the page
    a. `from django.http import HttpResponse`
    b.
    c.
    d. `def index(request):`
    e. `    return HttpResponse("Hello, world. You're at the polls index.")`
3. Now to actually see this views page on a browser, we must MAP it to a url and this can be done using a URL configuration, AKA, "urlconf". Urlconf are defined in each Django app, such as Polls, and they are specific Python named urls.py. So create a urls.py file in this app.
4. To define a urlconf for the polls app, add this code in the urls.py file:
    a. `from django.urls import path`
    b.
    c. `from . import views`
    d.
    e. `urlpatterns = [`
    f. `    path("", views.index, name="index"),`
    g. `]`
5. Now we must configure the root urlconf in the "mysite" project to include the urlconf in polls.url. Basically, we need to make sure the main urls page can include the polls page. This can be done by:
    a. **Add this line:** from django.urls import include, path. Adding this line allows this file to be able to include views pages from other apps
    b. **Add this line:** >>path("polls/", include("polls.urls")),<< (Don't forget the comma at the end. Everything inside the >><< should be included). This line is us adding the new views page via the urls configuration in the polls folder.
    c. mysite/urls.py should now look like this:

```
d. from django.contrib import admin
e. from django.urls import path
f. from django.urls import include, path
g.
h. urlpatterns = [
i.     path("polls/", include("polls.urls")),
j.     path('admin/', admin.site.urls),
k. ]
```

    l.
6. Recap: To create an app, we must
    a. First create the app (by running the command)

b. Then add views code for that app
c. To actually see the code on a browser, we must configure the views page to a url using “urlconfig”
d. Then we configure the root urlconfig in the “mysite” project to include the polls urlconfig. “Include” is a keyword here.

Database Setup: Link

1. Django is equipped with a default DB, SQLite. But for real project practice using more scalable DBs such as PostgreSQL
2. Go to mysite/settings.py and change time zone to “America/New_York”
    a. Making the timezone America/New_York as opposed to EST(winter) or EDT(summer) accounts for daylight savings time.
3. In the same settings.py file, you’ll see the INSTALLED_APPS list and they contain apps, all built in Django and some use at least one DB table. So to be able to use these apps’s DB data, we must create the DB tables, and so we create the tables in the DB using this command: >>python manage.py migrate<< Run this command in the djangotutorial folder

Creating DB models

1. A model is the single, definitive source of information about your data. It contains the essential fields and behaviors of the data you’re storing. Django follows the DRY Principle. The goal is to define your data model in one place and automatically derive things from it. This includes the migrations, which are derived from your models file. They form a history that Django uses to update your database schema to match your current models.
2. To create a DB model for tables for an app, you want to know your primary and foreign keys if necessary. In our tutorial example, we’ll need the Question and Choice Tables for our polls app.

```
a. class Question(models.Model):
b.     """
c.         This class represents a database table definition; The
   blueprint for all Question objects. A question has 2
   attributes:
d.         1. the text
e.         2. the publication date
f.     """
g.     question_text = models.CharField(max_length=200)
h.     pub_date = models.DateTimeField("date published")
i. ____________________________________________________________________
j.
k. class Choice(models.Model):
```

```
    """
        The blueprint for all Choice objects. A Choice has 2 attributes but is associated with a Question. A choice has 2 attributes:
        1. the text
        2. the # of votes
        (question) -> this is a foreign key to the Question class.
    """
    question = models.ForeignKey(Question, on_delete=models.CASCADE)
    choice_text = models.CharField(max_length=200)
    votes = models.IntegerField(default=0)
```

3. The DB will allow for the creation of new Question and Choice objects.

Adding apps to mysite/settings.py

1. We now need to add our polls app to the INSTALLED_APP list
2. We add this: >>“polls.apps.PollsConfig”,<< Add the quotes and the comma
3. It should look like this at the end
    a.
```
INSTALLED_APPS = [
    "polls.apps.PollsConfig",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]
```

Storing Migration

1. Once you’ve made these changes to the models and added the apps to mysite/settings.py you can now let Django know you’ve made changes to your models by running the makemigrations command like this: >>python manage.py makemigrations polls<<
2. The models.py changes are then stored as a migration. Migrations are how Django stores changes to your models which is your DB schema.
3. Recap for making model changes:
    a. Change your models (models.py)
    b. Run >>python manage.py makemigrations<< to create migrations for those changes
    c. Run >>python manage.py migrate<< to apply those changes to the DB

Check for project problems

1. A command to check for any problems in your project without making migrations: >>python manage.py check<<

Creating an Admin

1. Run python manage.py createsuperuser
2. Then navigate to polls/admin.py and add “from .models import Question” and then add this: “admin.site.register(Question)”
3. Enter username, email, and password
4. Then run the app again and navigate to http://127.0.0.1:8000/admin
5. Then you can create more Questions